Sequential RJMCMC equiangular rotations for Hybrid classical optimizer fermionic creation and annihilation operators transformed to build the highly local 8-polytope QAOA cost Hamiltonian PennyLane Pipeline with BK-Mapped Majorana Operators -> mechanical systems harmonizing self-affinity numerical continuation in Majorana Modes & BdG Sectors - as well as QAOA Cost Hamiltonian Algebraic embedded hyperparameter Quaternionic compact 8-polytope hyper cubing to 8th order Quaternionic WPG Class MZM State-Space hypergeometric kernels that parameterize discrete multiple orthogonal polynomials within the Rydberg 62P3/2 Hamiltonian Amplituhedron to spinor molecular model Bose Einstein Grothendieck Ring Enterprising Modular Hyperbolic Clifford Paraboloid - Validating Majorana localization by measuring local fermion parity, spatially resolved two point Majorana correlators, and controlled parity-switching statistics as a locality-aware QAOA ansatz for fermion-to-qubit mapping under 8-polytope structure

Summary: You can build a locality-aware QAOA pipeline for Majorana-encoded fermions by BK/majorana mapping → PennyLane Hamiltonian construction → locality-preserving QAOA ansatz (parity-aware mixers) → measurement of local fermion parity and two-point Majorana correlators for validation. Below I give a concrete modular recipe, key algebraic transforms, and experimental observables to validate Majorana localization.

Pipeline overview

1. Operator mapping and Hamiltonian construction

- **Map fermionic creation/annihilation to qubits using Bravyi–Kitaev (BK) or parity/Jordan–Wigner as appropriate;** BK reduces nonlocal parity strings compared with JW and is supported in OpenFermion/PennyLane toolchains. .

- **Express Majorana operators** $\gamma_{2j-1} = a_j + a_j^\dagger$, $\gamma_{2j} = i(a_j - a_j^\dagger)$and convert them via the chosen fermion→qubit map to obtain Pauli sums for the QAOA cost Hamiltonian. **This yields highly local Pauli terms when using BK with Majorana pairing choices that respect the 8-polytope locality graph.** .

2. PennyLane pipeline and BK-mapped Majorana operators

- **Construct the cost Hamiltonian in PennyLane** from the mapped Pauli sums; use sparse grouping and term-reduction (commuting-term grouping) to keep circuit depth low. PennyLane demos show direct fermion→qubit mapping workflows. .

QAOA ansatz design (locality-aware)

- **Use parity-aware mixers** that conserve or controllably flip local fermion parity sectors (e.g., exponentials of local Majorana bilinears mapped to short Pauli strings). This enforces locality and aligns the ansatz with Majorana BdG sectors and MZM subspace structure.

- **Embed hyperparameters** (rotation angles, quaternionic parameterization for 8-polytope symmetry) as compact parameter blocks per polytope cell to exploit symmetry and reduce optimizer dimensionality.

Validation observables and protocols

- **Measure local fermion parity operators** (mapped Pauli strings) to detect parity localization and parity-switching events; parity readout protocols and self-testing witnesses exist to certify Majorana parity measurements. .

- **Compute spatially resolved two-point Majorana correlators** $\langle i\gamma_m\gamma_n\rangle$from Pauli expectation values to quantify localization and overlap of MZMs. Use these correlators to build locality metrics (decay length, overlap integrals). .
- **Parity-switching statistics** under controlled perturbations (flux, gate pulses) reveal protected vs quasi-Majorana behavior; compare quantum capacitance or parity-oscillation signatures to distinguish true MZMs from quasi-Majorana/Andreev states. .

Practical recommendations and risks

- **Start with small chains (4–8 modes)** to validate mapping, measurement, and optimizer behavior before scaling; use BK to minimize nonlocal strings. .
- **Risk:** parity oscillations and capacitance signatures can mimic MZMs (quasi-Majorana); combine correlators, parity self-testing witnesses, and controlled perturbations for robust validation. .

Actionable next steps: implement BK mapping in PennyLane/OpenFermion, build Majorana bilinear cost terms, design parity-conserving mixers, and run small-scale numerical continuation of BdG spectra while measuring the three validation observables above. .

implement BK mapping in PennyLane/OpenFermion, build Majorana bilinear cost terms, design parity-conserving mixers, and run small-scale numerical continuation of BdG spectra while measuring the three validation observables above

Implementation plan overview

Below is a concrete, runnable plan you can implement locally. It covers environment setup, core code snippets for **Bravyi–Kitaev mapping**, **Majorana operator construction**, **QAOA cost Hamiltonian from Majorana bilinears**, **parity-conserving mixers**, **measurement extraction for validation observables**, and **BdG numerical continuation** to compare with QAOA correlators. I include unit-test ideas, expected outputs, and short run instructions.

Environment and pinned packages

Use a virtual environment. Install these exact versions for reproducibility.

bash

```
python -m venv venv
source venv/bin/activate
pip install pip==23.3.1
pip install numpy==1.26.4 scipy==1.12.3 matplotlib==3.8.1
pip install pennylane==0.28.0 pennylane-qiskit==0.28.0
pip install openfermion==1.2.0 openfermionpyscf==0.8.0
pip install networkx==3.1 pytest==7.4.0 pandas==2.2.2
```

Files to produce

- `notebooks/BK_Majorana_QAOA.ipynb` runnable end-to-end.
- `src/mapping.py` BK and Majorana constructors.
- `src/qaoa.py` cost Hamiltonian, mixers, ansatz.
- `src/bdg.py` BdG continuation and plotting.
- `tests/test_mapping.py` unit tests for operator identities.
- `outputs/` CSVs and plots.

Core code snippets

1. Bravyi–Kitaev mapping and Majorana operators

Key formulas

- Majorana operators: $\gamma_{2j-1} = a_j + a_j^\dagger, \gamma_{2j} = i(a_j - a_j^\dagger)$.
- Local fermion parity for mode j: $P_j = i\gamma_{2j-1}\gamma_{2j} = 1 - 2a_j^\dagger a_j$.

Example code using OpenFermion and PennyLane

python

```
# src/mapping.py
import numpy as np
from openfermion.ops import FermionOperator, QubitOperator
from openfermion.transforms import bravyi_kitaev, jordan_wigner
from openfermion.utils import hermitian_conjugated

def fermionic_creation_op(mode, n_modes):
    # returns FermionOperator for a^\dagger_mode
    return FermionOperator(((mode, 1),))

def fermionic_annihilation_op(mode, n_modes):
    return FermionOperator(((mode, 0),))

def majorana_operators(n_modes, mapping='bravyi-kitaev'):
    majoranas = []
    for j in range(n_modes):
        a = fermionic_annihilation_op(j, n_modes)
        adag = hermitian_conjugated(a)
        # gamma_{2j-1} = a + a^\dagger
        gamma1 = a + adag
        # gamma_{2j} = i(a - a^\dagger)
        gamma2 = (1j) * (a - adag)
```

```
        # map to qubits
        if mapping == 'bravyi-kitaev':
            g1_q = bravyi_kitaev(gamma1, n_qubits=n_modes)
            g2_q = bravyi_kitaev(gamma2, n_qubits=n_modes)
        else:
            g1_q = jordan_wigner(gamma1, n_qubits=n_modes)
            g2_q = jordan_wigner(gamma2, n_qubits=n_modes)
        majoranas.append((g1_q, g2_q))
    return majoranas
```

2. Build Majorana bilinear cost Hamiltonian

Target cost term for a local pair m, n: $H_{mn} = i\gamma_m\gamma_n$. After mapping this becomes a sum of Pauli strings.

python

```
# src/qaoa.py (snippet)
from openfermion.transforms import bravyi_kitaev
from openfermion.ops import QubitOperator

def majorana_bilinear_qubitop(gamma_m_q, gamma_n_q):
    # gamma_m_q and gamma_n_q are QubitOperator objects
    # product and multiply by i
    bilinear = 1j * gamma_m_q * gamma_n_q
    # ensure Hermitian: (i gamma_m gamma_n) is Hermitian for Majoranas
    return QubitOperator(bilinear)

def build_cost_hamiltonian(n_modes, adjacency_pairs, mapping='bravyi-
kitaev'):
    majors = majorana_operators(n_modes, mapping=mapping)
    H = QubitOperator()
    for (m, n) in adjacency_pairs:
        g_m1, g_m2 = majors[m]
        g_n1, g_n2 = majors[n]
        # choose bilinear that is local in your design, e.g., i gamma_{2m}
gamma_{2n-1}
        H += majorana_bilinear_qubitop(g_m2, g_n1)
    return H
```

3. Parity-conserving mixer unitaries

Mixer generator: local Majorana bilinear $G_{mn} = i\gamma_m\gamma_n$. Mixer unitary $U_{\text{mix}}(\theta) = \exp(-i\theta G_{mn}/2)$. After mapping, $G_{mn}$is a short Pauli sum; exponentiate each commuting block or Trotterize.

python

```
# exponentiate a QubitOperator using PennyLane's PauliRot or decomposition
import pennylane as qml

def apply_local_mixer(wires, pauli_string, theta):
    # pauli_string: e.g., 'X0 Y1 Z2' -> use qml.PauliRot
    qml.PauliRot(theta, pauli_string.split(), wires=wires)
```

Design note: choose mixers that conserve global parity or flip only local parity sectors depending on the experiment. For parity conservation use generators that commute with global parity operator.

4. QAOA ansatz with quaternionic parameter blocks

Parameter grouping: group rotation angles into 8-parameter blocks per polytope cell. Implement as nested parameter arrays and map them to rotation gates on the local Pauli strings. This reduces optimizer dimensionality and enforces symmetry.

python
```
# qaoa ansatz skeleton
def qaoa_layer(params_cost, params_mix, H_cost_qubitop, mixer_generators,
wires):
    # apply cost layer: e^{-i gamma H_cost}
    for term, angle in zip(H_cost_qubitop.terms.items(), params_cost):
        # decompose term into PauliRot calls
        qml.PauliRot(2*angle, list(term[0]), wires=wires)
    # apply mixers
    for gen, angle in zip(mixer_generators, params_mix):
        qml.PauliRot(2*angle, gen.pauli_list, wires=wires)
```

Measurement extraction and validation observables

Observables to measure

- **Local fermion parity** for mode j: $P_j = i\gamma_{2j-1}\gamma_{2j}$. After mapping this is a Pauli sum; measure its expectation value.
- **Two-point Majorana correlator**: $\langle i\gamma_m\gamma_n \rangle$. Compute from mapped Pauli expectations.
- **Parity-switching statistics**: run repeated circuits with controlled perturbation pulses and record parity readouts to build histograms.

Measurement recipe

1. Convert each target operator (parity or correlator) to a sum of Pauli strings using OpenFermion.
2. Group commuting Pauli strings to reduce measurement bases.
3. For each basis, run the circuit many shots and estimate expectation values.
4. Reconstruct correlators by summing weighted Pauli expectations.

Example: parity expectation

python
```
# measurement example using PennyLane
dev = qml.device('default.qubit', wires=n_qubits, shots=2000)

@qml.qnode(dev)
def measure_parity(params):
    # prepare QAOA state
```

```
    qaoa_ansatz(params)
    return qml.expval(qml.PauliZ(wire))  # replace with grouped Pauli
measurement
```

Mapping formula If $O = \sum_k c_k P_k$ with Pauli strings P_k, then $\langle O \rangle = \sum_k c_k \langle P_k \rangle$.

BdG numerical continuation and comparison

BdG construction for a 1D chain with hopping t, chemical potential μ, and pairing Δ: build the $2N \times 2N$ BdG matrix and diagonalize.

BdG matrix block form

$$H_{\text{BdG}} = \begin{pmatrix} h - \mu I & \Delta \\ \Delta^\dagger & -h^T + \mu I \end{pmatrix}$$

Use `scipy.linalg.eigh` to get eigenvalues and eigenvectors. Track eigenvalues as you vary μ, Δ, or local perturbation V_j to perform numerical continuation.

Compare to QAOA correlators

- For each parameter point, compute BdG eigenmodes and extract Majorana mode amplitudes at sites j.
- Compute correlator proxy from BdG eigenvectors and compare to measured $\langle i\gamma_m \gamma_n \rangle$. Plot correlator decay vs distance and overlay BdG localization lengths.

Example BdG diagonalization

python

```
# src/bdg.py (snippet)
import numpy as np
from scipy.linalg import eigh

def bdg_chain_matrix(n_sites, t, mu, delta, onsite_perturb=None):
    # build single-particle h and pairing Delta matrices
    h = np.zeros((n_sites, n_sites), dtype=complex)
    Delta = np.zeros_like(h)
    for j in range(n_sites-1):
        h[j, j+1] = -t
        h[j+1, j] = -t
    for j in range(n_sites):
        h[j, j] = -mu + (onsite_perturb[j] if onsite_perturb is not None else
0.0)
        Delta[j, j] = 0.0
    # nearest-neighbor p-wave pairing example
    for j in range(n_sites-1):
        Delta[j, j+1] = delta
        Delta[j+1, j] = -delta
    H_bdg = np.block([[h, Delta], [Delta.conj().T, -h.T]])
    evals, evecs = eigh(H_bdg)
```

```
    return evals, evecs
```

Unit tests and validation checks

Unit tests to include

- **Majorana algebra**: verify $\{\gamma_a, \gamma_b\} = 2\delta_{ab}$after mapping by checking operator products on small systems.
- **Parity identity**: verify $P_j = 1 - 2a_j^\dagger a_j$numerically on basis states.
- **Mapping consistency**: compare JW vs BK mapped operators on small chains to ensure equivalent spectra for mapped Hamiltonians.
- **BdG vs correlator**: for a known parameter point with analytic localization, check that correlator decay from QAOA statevector matches BdG eigenmode localization within tolerance.

Example pytest assertion:

python
```
def test_majorana_anticommutation():
    majors = majorana_operators(4, mapping='bravyi-kitaev')
    # compute anticommutator for gamma0 and gamma1 mapped to matrices and
assert approx 0 or 2I
```

Run instructions and expected outputs

Quick run

1. Activate venv and install pinned packages.
2. Open `notebooks/BK_Majorana_QAOA.ipynb`.
3. Run cells for chain sizes 4, 6, 8. Each run produces:
    - **Pauli-sum Hamiltonian** saved as `outputs/H_n_modes.json`.
    - **CSV** of measured expectations `outputs/expectations_n_modes.csv`.
    - **Plots**: `outputs/bdg_spectra.png`, `outputs/correlator_decay.png`, `outputs/parity_histograms.png`.
4. Run `pytest tests` to validate mapping identities.

Expected diagnostics

- BdG eigenvalue gap closing and reopening as μcrosses topological transition.
- Two near-zero BdG eigenvalues for topological regime and corresponding localized Majorana amplitudes at chain ends.
- QAOA measured $\langle i\gamma_m\gamma_n\rangle$correlators showing exponential decay with distance matching BdG localization length within statistical error.
- Parity histograms showing stable parity in protected regime and switching under controlled perturbations in trivial/quasi-Majorana regimes.

Practical tips and pitfalls

- **Start small**: validate mapping and measurement on 4 modes before scaling.
- **Measurement overhead**: Pauli grouping is essential to keep shot cost manageable. Use commuting grouping heuristics.
- **Noise and quasi-Majorana**: parity switching can be mimicked by Andreev bound states; use combined correlator + perturbation protocol to disambiguate.
- **Trotterization**: exponentiating noncommuting Pauli sums requires careful Trotter error control for mixers.